Informe Técnico de Arquitectura — Simulador de Kernel y Gestor de Procesos (Fase I)

Nov 11, 2026 · @Aarón

Universidad Católica Andrés Bello · Facultad de Ingeniería · Escuela de Ingeniería Informática

Cátedra: Sistemas Operativos · Profesor: Marcel J. Castro G.

Proyecto Fase I: Simulador de Kernel y Gestor de Procesos en Consola (Shell & Scheduling Engine)

Integrante	Rol / módulo	Cédula
Aaron Zarraga	Dev 1: PCB y motor de scheduling	C.I. 31.867.135
Sebastianno Verrocchi	Dev 2: CLI y validaciones	C.I. 31.867.118
Carlos La Verde	Dev 3: infraestructura, hilos POSIX y compilación	C.I. 31.985.243

Repositorio: github.com/aaronzarragon/scheduler-fase1 · Versión entregada: `v1.0.1-fase1` · Fecha de entrega: 11 de noviembre de 2026

1. Introducción

El simulador reproduce en consola el núcleo de planificación de un sistema operativo: crea procesos, mantiene su PCB y reparte un CPU simulado entre ellos con Round Robin o SRTF. Está escrito en C++17 y usa hilos POSIX (`pthread`) con exclusión mutua (`pthread_mutex_t`).

El objetivo de la Fase I es separar dos responsabilidades que corren a la vez: una Shell que atiende al operador sin bloquearse y un motor que avanza el reloj y ejecuta la política de planificación. El sistema acepta procesos interactivos (`create`) y cargas por lotes (`load`), y reporta el diagrama de Gantt y las métricas de espera, retorno, respuesta y uso de CPU.

2. Arquitectura general

El programa corre con dos hilos POSIX que nunca se llaman entre sí: se comunican solo a través de un estado global compartido, y todo acceso a ese estado ocurre con `g_mutex` tomado.

- **Hilo CLI** (`cliThread`): lee una línea, la divide en tokens, valida cantidad, tipo y rango de cada argumento y ejecuta el comando. Ante una entrada inválida imprime un error y vuelve a pedir la siguiente; nunca termina por un error.
- **Hilo Motor** (`schedulerThread`): mientras el motor está activo, ejecuta un tick de CPU por ciclo y luego duerme `g_speed_ms` milisegundos reales. Cada tick admite llegadas, aplica la política, consume un tick de ráfaga y registra el Gantt.
- **Hilo principal** (`main`): crea los dos hilos, espera a que la CLI termine (`exit` o fin de entrada), señaliza la salida al motor, lo espera con `pthread_join` y destruye el mutex.

El tiempo del sistema es un contador entero (`g_clock`) que sube en 1 por cada ciclo de CPU. El retardo real solo controla la velocidad de la simulación; ninguna métrica usa el reloj de la computadora.

3. Estructura modular

El código está en `scheduler/`, separado en cabeceras (`include/`) y fuentes (`src/`); cada módulo tiene un responsable según la división de trabajo.

Archivo	Contenido	Responsable
`include/pcb.h`	`enum class ProcessState`, `struct PCB` y `stateToString()`	Aaron
`include/shared_state.h`	Estado global compartido (`extern`), `GanttEntry`, `SchedAlgo` y la regla de uso del mutex	Carlos
`src/shared_state.cpp`	Definición e inicialización de las variables globales, `findPCB()` y `countFinished()`	Aaron
`include/scheduler.h`	Interfaz del motor: `schedulerThread()` y `selectNext()`	Aaron

Archivo	Contenido	Responsable
`src/scheduler.cpp`	Ciclo del tick, despacho, RR, SRTF, desapropiación, Gantt y cálculo de métricas al terminar	Aaron
`include/cli.h`	Interfaz de la Shell: `cliThread()`	Sebastianno
`src/cli.cpp`	Lectura, tokenizado, validación y ejecución de los 12 comandos	Sebastianno
`src/main.cpp`	Creación y `join` de los hilos, cierre limpio y destrucción del mutex	Carlos
`Makefile`	Compilación con `g++ -std=c++17 -Wall -Wextra -pthread`	Carlos
`workload_test.txt`	Traza de prueba del Escenario 2	Carlos

Las variables compartidas son: `g_processes` (todos los PCB), `g_ready_queue` (cola de PIDs listos para RR), `g_clock`, `g_algo`, `g_quantum`, `g_speed_ms`, `g_running`, `g_exit`, `g_gantt` y `g_next_pid`.

4. Modelo del proceso (PCB)

Cada proceso es un `struct PCB` guardado en `g_processes`; el PID es único y autoincremental desde 1.

Campo	Significado
`pid`, `name`, `priority`	Identificador, nombre sin espacios y prioridad de 1 a 5 (por defecto 3)
`state`	`NEW`, `READY`, `RUNNING`, `BLOCKED` o `TERMINATED`
`burst_initial`, `burst_remaining`	Ráfaga total y ticks que faltan
`arrival_time`	Tick de llegada al sistema
`start_time`	Tick del primer tick de CPU consumido (−1 si nunca ejecutó)
`finish_time`	Tick en que terminó (−1 si no ha terminado)

Campo	Significado
`waiting_time`, `turnaround_time`, `response_time`	Métricas WT, TAT y RT, calculadas al terminar

Transiciones de estado:

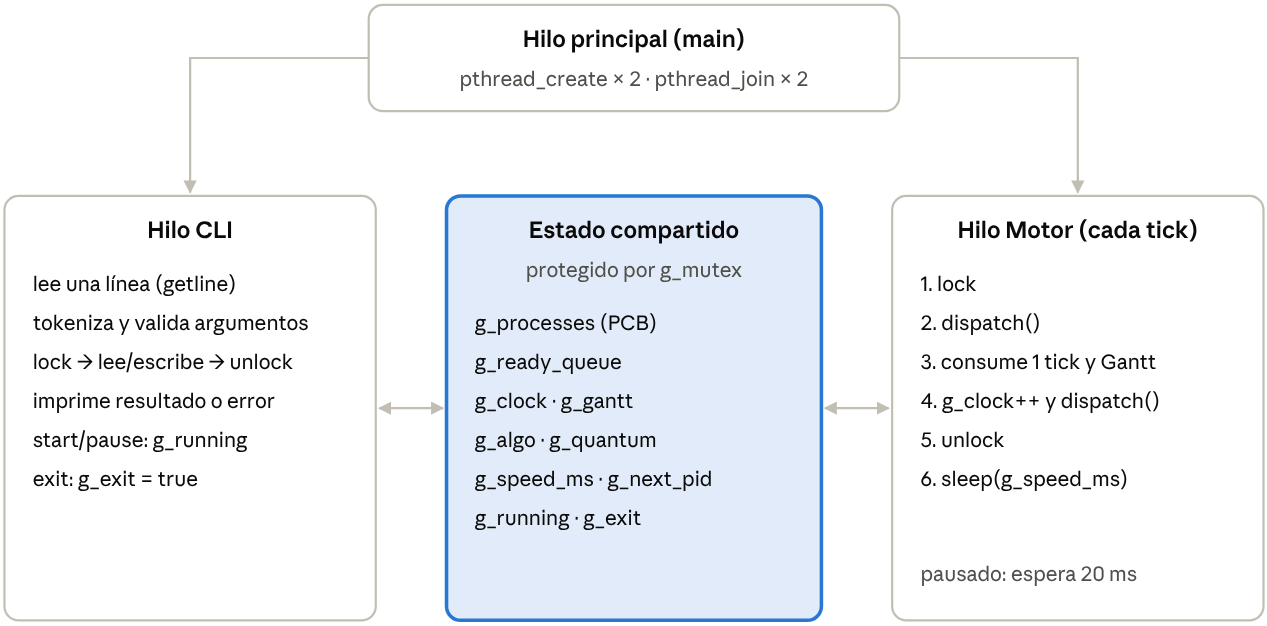
1. `create` construye el PCB con llegada en el tick actual y lo deja en READY, dentro de la cola de listos.
2. `load` deja en NEW los procesos cuyo `arrival_time` es futuro; el motor los pasa a READY al llegar ese tick. Los que ya llegaron entran directo a READY.
3. El despacho pasa un proceso de READY a RUNNING.
4. RUNNING vuelve a READY cuando vence el quantum (RR) o cuando otro proceso tiene menor ráfaga restante (SRTF).
5. RUNNING pasa a TERMINATED cuando su ráfaga restante llega a 0.

El estado `BLOCKED` está definido en el modelo, pero la Fase I no simula operaciones de E/S, así que ningún proceso entra en él. Al pausar el motor, los procesos conservan su estado y el reloj se detiene.

5. Hilos y sincronización

Un único mutex (`g_mutex`) protege todo el estado compartido, y el motor nunca duerme con él tomado.

Los hilos CLI y Motor solo se comunican por el estado protegido con g_mutex



Toda lectura o escritura del estado ocurre entre pthread_mutex_lock y pthread_mutex_unlock.
El motor duerme fuera del mutex, así la CLI responde de inmediato aunque el tick dure 5 s.

Arquitectura de hilos y sincronización · 3 hilos, 1 mutex

Secciones críticas. Cada comando de la CLI toma el mutex solo mientras lee o modifica el estado, y lo suelta antes de esperar la siguiente entrada. Un tick completo del motor (despacho, consumo, Gantt, avance del reloj) ocurre dentro de una sola sección crítica, así que `ps`, `pcb`, `gantt` y `stats` ven siempre un estado consistente entre ticks, nunca uno a medias. `load` inserta todo el lote en una sola sección crítica.

Sin interbloqueos. Hay un solo mutex y ninguna función lo toma dos veces, así que no puede haber espera circular. Las funciones auxiliares `findPCB()` y `countFinished()` se llaman siempre con el mutex ya tomado.

Control del motor. `start` y `pause` solo cambian `g_running`. Mientras está pausado o no queda trabajo, el motor revisa la bandera cada 20 ms sin avanzar el reloj.

Cierre limpio. `exit` (o el fin de la entrada) pone `g_exit = true` y termina el hilo CLI. `main` lo espera con `pthread_join`, confirma la señal de salida, espera al motor con `pthread_join` y destruye el mutex con `pthread_mutex_destroy`. Los PCB viven en contenedores estándar (`std::vector`, `std::deque`) que liberan su memoria al terminar el programa.

La compilación con ThreadSanitizer no reportó condiciones de carrera (sección 10).

6. Planificación de CPU

Toda decisión de planificación la toma la función `dispatch()`, que se ejecuta dos veces por tick con el mutex tomado: al inicio, antes de consumir el tick, y al final, después de avanzar el reloj. La llamada del final hace que, mientras el motor duerme, `ps` ya muestre como RUNNING al proceso que tiene la CPU.

`dispatch()` hace tres pasos en orden y es idempotente:

1. **Llegadas:** los procesos NEW con `arrival_time <= g_clock` pasan a READY y entran al final de la cola.
2. **Desapropiación (solo SRTF):** si un proceso READY tiene una ráfaga restante estrictamente menor que la del proceso RUNNING, este vuelve a READY.
3. **Selección:** si la CPU está libre, se elige el siguiente proceso según la política activa y pasa a RUNNING.

Round Robin. Se toma el primer PID READY de `g_ready_queue`. El proceso ejecuta como máximo `quantum` ticks seguidos; al vencer el quantum vuelve al final de la cola. Las llegadas del mismo tick se encolan después del proceso desalojado. El quantum por defecto es 2.

SRTF. Se elige el proceso READY con menor `burst_remaining`; ante un empate gana el que se creó primero. Como la desapropiación se evalúa en cada tick, un proceso que llega con menos ráfaga restante toma la CPU en el mismo tick de su llegada. Así, en el Escenario 2, Job_B (BT = 2) desaloja a Job_A (BT restante = 3) en t = 1.

Cambio en caliente. `set_sched` solo cambia `g_algo` (y `g_quantum` para RR). El motor lee la política en el siguiente despacho, así que el cambio toma efecto en el siguiente tick, esté el motor corriendo o en pausa. La cola READY se mantiene sin PIDs repetidos para que RR siga siendo justo después de un cambio.

Diagrama de Gantt. Cada tick agrega el intervalo `[t, t+1)` con el nombre del proceso ejecutado, o `IDLE` si no había ninguno listo. Si el intervalo es del mismo proceso que el anterior, se fusiona con él. Cuando todos los procesos terminaron, el reloj se congela para no acumular IDLE al final; si no hay ningún proceso, `start` avanza el reloj en IDLE.

Métricas. Al terminar un proceso se calculan, en ticks:

- **Tiempo de retorno:** TAT = t_fin − t_llegada
- **Tiempo de espera:** WT = TAT − ráfaga_inicial
- **Tiempo de respuesta:** RT = t_primer_tick − t_llegada

`stats` promedia WT, TAT y RT sobre los procesos terminados, y calcula el uso de CPU como los ticks no IDLE del Gantt entre el reloj total:

Uso de CPU = (ticks ejecutando ÷ g_clock) × 100 %

`pcb` muestra el WT parcial de un proceso que aún no termina: ticks transcurridos desde su llegada menos los ticks que ya ejecutó.

7. Modo por lotes (`load`)

`load` valida todo el archivo antes de insertar un solo proceso: si una línea es inválida, informa el número de línea y el motivo, y no carga nada.

Formato esperado, un proceso por línea:

```
# NOMBRE, BURST_TIME, PRIORIDAD, ARRIVAL_TIME
Job_A, 4, 2, 0
Job_B, 2, 1, 1
Job_C, 1, 3, 3
```

El proceso tiene dos fases:

1. **Validación.** Se verifica que la ruta exista y sea un archivo regular, no una carpeta. Se ignoran las líneas vacías y los comentarios (`#`), y se aceptan archivos con fin de línea de Windows (CRLF) o con BOM UTF-8. Cada línea debe tener exactamente 4 campos: nombre no vacío y sin espacios, `BURST_TIME` entero > 0, `PRIORIDAD` entera de 1 a 5 y `ARRIVAL_TIME` entero ⩾ 0. Un archivo sin procesos también se rechaza.
2. **Inserción.** Todos los PCB se insertan dentro de una sola sección crítica, así el motor nunca ve un lote a medias. Los procesos que ya llegaron quedan en READY y los demás en NEW, a la espera de su `arrival_time` .

8. Shell (CLI) y validación de entradas

La Shell reconoce 12 comandos, distingue mayúsculas de minúsculas y responde a toda entrada inválida con un mensaje del tipo de error, sin cerrarse ni bloquearse.

Comando	Validaciones
`create <nombre> <burst> [prioridad]`	2 o 3 argumentos; burst entero > 0; prioridad entera de 1 a 5 (por defecto 3)

Comando	Validaciones
`load <archivo>`	1 argumento; existencia y formato del archivo (sección 7)
`ps`, `start`, `pause`, `gantt`, `stats`, `help`, `exit`	Ningún argumento
`pcb <PID>`	1 argumento; entero positivo; el PID debe existir
`set_sched <RR|SRTF> [quantum]`	Algoritmo exactamente `RR` o `SRTF`; quantum entero > 0, solo con RR (por defecto 2)
`set_speed <ms_por_tick>`	1 argumento; entero entre 50 y 5000

Los números se validan con `std::stoi` comprobando que se consumió todo el texto, así que `3.5`, `5abc` o un valor que desborda `int` se rechazan.

Categoría	Cuándo aparece	Ejemplo
`[ERROR DE SINTAXIS]`	Faltan o sobran argumentos	`create P1`
`[ERROR DE TIPO]`	El valor no es un entero válido o no es positivo	`pcb abc`, `create P1 0`
`[ERROR DE RANGO]`	El número está fuera del rango permitido	`create P1 5 9`, `set_speed 99999`
`[ERROR DE PARAMETRO]`	El valor no es una de las opciones válidas	`set_sched FIFO`
`[ERROR DE FORMATO]`	Una línea del archivo de `load` es inválida	`Job_A, abc, 2, 0`
`[ERROR]`	Archivo inexistente, PID inexistente o comando desconocido	`pcb 999`, `load x.txt`

9. Gestión del proyecto en GitHub

El equipo usó Gitflow: cada integrante subió su módulo en su propia rama `feature/*` y lo integró a `develop` mediante un Pull Request. `main` solo recibe versiones entregables.

- **Ramas:** `main` (entregas), `develop` (integración), `feature/pcb-scheduler`, `feature/cli-commands`, `feature/core-infra` y las ramas `fix/*` de correcciones. `main` y `develop` están protegidas: solo aceptan cambios por Pull Request.
- **Commits por integrante:** cada módulo lleva la autoría de su responsable. La cuenta de Carlos estaba bloqueada el día de la integración, así que Aaron subió su código con Carlos como autor del commit (Git registra por separado autor y committer).
- **Issues:** uno por módulo y uno por cada corrección o entregable, con etiquetas `modulo:scheduler`, `modulo:cli`, `modulo:infra`, `tipo:feature`, `tipo:bug`, `tipo:docs` y `tipo:gestion`, asignados a su responsable.
- **Kanban:** tablero de GitHub Projects con columnas To Do, In Progress y Done, que agrupa todos los issues.
- **Versiones:** etiquetas `v1.0-fase1` y `v1.0.1-fase1` sobre `main`, publicadas como Releases.

PR	Rama	Contenido	Autor
#1	`feature/pcb-scheduler`	PCB y motor RR/SRTF	Aaron
#2	`feature/cli-commands`	Shell y validaciones	Sebastianno
#3	`feature/core-infra`	Estado compartido, hilos, Makefile y workload	Carlos
#4	`fix/reloj-idle-y-load`	Reloj congelado al terminar; `load` admite llegadas en t = 0	Aaron
#5	`develop` → `main`	Release v1.0	Aaron
#6	`fix/validaciones-y-despacho`	Despacho al final del tick, `load` estricto y mutex en `pause`	Aaron
#16	`develop` → `main`	Release v1.0.1	Aaron

10. Pruebas y resultados

Los dos escenarios del enunciado producen exactamente el Gantt esperado, y ninguna de las entradas inválidas probadas cerró ni congeló el simulador. Las pruebas se corrieron en WSL (Ubuntu, g++ 15.2) a 50 ms por tick.

Prueba	Resultado obtenido
Escenario 1: RR, quantum 2, P1 (5), P2 (3), P3 (2)	`[0-2s: P1] -> [2-4s: P2] -> [4-6s: P3] -> [6-8s: P1] -> [8-9s: P2] -> [9-10s: P1]`; WT 5.00, TAT 8.33, RT 2.00, CPU 100.00%
Escenario 1 con `pause`, `pcb 1`, `set_sched SRTF` y `start`	RR hasta la pausa y SRTF después; 3/3 procesos terminados, CPU 100.00%
Escenario 2: SRTF con `workload_test.txt`	`[0-1s: Job_A] -> [1-3s: Job_B] -> [3-4s: Job_C] -> [4-7s: Job_A]`; WT 1.00, TAT 3.33, RT 0.00, CPU 100.00%
12 errores de la Guía de Usuario y unas 60 entradas inválidas adicionales	Cada una con su categoría de error; el simulador sigue esperando comandos
Archivos de `load` mal formados (campos de más o de menos, decimales, negativos, carpeta, vacío)	Error con número de línea; no se carga ningún proceso
RR → SRTF → RR en caliente con 4 procesos	Cada proceso ejecutó exactamente su ráfaga; 4/4 terminados
108 muestras de `ps` durante RR y SRTF	En todas hay exactamente 1 proceso RUNNING mientras queda trabajo
Compilación con ThreadSanitizer (`-fsanitize=thread`) y `pause` / `start` / `set_sched` en caliente	Ninguna condición de carrera reportada
Compilación con `-Wall -Wextra`	Sin advertencias

El Escenario 1 da WT 5.00 porque P1 termina en t = 10, P2 en t = 9 y P3 en t = 6: sus tiempos de espera son 5, 6 y 4 ticks.

11. Compilación y ejecución

El proyecto compila con `make` en cualquier entorno POSIX con g++ y soporte C++17 (Linux, WSL o macOS).

```
git clone https://github.com/aaronzarragon/scheduler-fase1.git
cd scheduler-fase1/scheduler
make
./scheduler
```

`make clean` borra los objetos y el ejecutable. En MSYS2 UCRT64 se usa `mingw32-make` en lugar de `make`.

12. Referencias bibliográficas

1. Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). Wiley. Capítulos 3 (procesos), 4 (hilos), 5 (planificación de CPU) y 6 (sincronización).
2. Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.ª ed.). Pearson. Capítulo 2 (procesos e hilos).
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9.ª ed.). Pearson. Capítulo 9 (planificación uniprocesador).
4. Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.
5. IEEE y The Open Group (2018). *IEEE Std 1003.1-2017 (POSIX.1-2017)*: `pthread_create`, `pthread_join`, `pthread_mutex_lock`. pubs.opengroup.org
6. *cppreference.com*: `std::this_thread::sleep_for`, `std::stoi`, `std::filesystem`. en.cppreference.com
7. Universidad Católica Andrés Bello, Cátedra de Sistemas Operativos (2026). *Proyecto Fase I: Simulador de Kernel y Gestor de Procesos en Consola* y *Guía de Usuario*.

Anexo A. Declaración de uso de Inteligencia Artificial

De conformidad con lo establecido en la Sección 9 del enunciado del proyecto, el equipo declara el uso de herramientas de Inteligencia Artificial de la siguiente manera.

A.1 Herramientas utilizadas

- **Claude (Anthropic)**, modelo Claude Sonnet 4.6, interfaz web claude.ai
- **Claude Code (Anthropic)**, modelo Claude Opus 5.5, extensión de VS Code, 10 y 11 de noviembre de 2026: configuración del repositorio y del flujo Gitflow, revisión del código contra el enunciado, pruebas, corrección de errores (PR #4 y #6), Issues y tablero Kanban, y borrador de este informe.

A.2 Rol asignado a la herramienta

Tutor socrático para entender los conceptos y asistente de depuración en la integración: la herramienta propuso y escribió correcciones, que el equipo revisó, probó y aprobó mediante Pull Requests antes de integrarlas.

A.3 Prompts principales

- "Cómo descargo WSL según la guía de usuario."
- "Crear el repo con Gitflow para que cada integrante suba su parte por commits."
- "Revisa las validaciones de errores del código y pruébalo las veces que sean necesarias hasta saber que funciona todo."
- "Haz el informe técnico, exceptuando la parte del anexo de IA."

A.4 Proceso de validación de los resultados

- **Revisión línea a línea:** cada función fue leída y comprendida por al menos un integrante antes de incorporarla al repositorio.
- **Pruebas manuales:** se ejecutaron los dos escenarios del enunciado verificando que la salida del Gantt coincida exactamente con la esperada.
- **Pruebas de errores:** se probaron todos los casos de error confirmando que el sistema no colapsa.
- **Comprensión individual:** cada integrante puede explicar el funcionamiento de cada módulo.
- **Integración revisada:** el código aportado por la IA se integró solo mediante Pull Requests que el equipo revisó; se verificó con los escenarios del enunciado y con pruebas de entradas inválidas, y cada integrante estudió su módulo para poder explicarlo.